

Smart Metro Ticket Booking System

Project Documentation

(Built on the ServiceNow Now Platform)

Project Name	Smart Metro Ticket Booking System
Team	
Platform	ServiceNow (Now Platform)
Document Type	Final Project / FSD Documentation

Table of Contents

1. INTRODUCTION

- 1.1 Project Overview
- 1.2 Purpose

2. IDEATION PHASE

- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. REQUIREMENT ANALYSIS

- 3.1 Customer Journey Map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. PROJECT DESIGN

- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

- 5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

- 6.1 Functional Testing (UAT)
- 6.2 Performance Testing

7. RESULTS

- 7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

- Source Code
- GitHub Link
- Project Demo Link

1. INTRODUCTION

1.1 Project Overview

Metro rail is one of the most widely used modes of urban public transport because it is fast, affordable, and reduces road congestion. However, in most systems, buying a metro ticket still depends heavily on manual counters, token machines, or partially digital kiosks located inside the station. Commuters are required to physically visit the station, stand in a queue, choose the source and destination stations, pay in cash or card, and collect a physical token or paper ticket before they are allowed onto the platform.

The **Smart Metro Ticket Booking System** is a solution built entirely on the ServiceNow (Now Platform) that digitizes this entire journey — from station selection and fare calculation to digital payment, QR-ticket generation, and real-time gate validation — for four personas: Passengers, Station Managers, the Metro Operations Team, and IT Admins.

1.2 Purpose

The purpose of this document is to consolidate the complete project lifecycle — ideation, requirement analysis, design, planning, testing, and results — into a single Functional Specification / Project Report. It serves as a single reference describing the problem being solved, the proposed ServiceNow-based solution, its architecture and technology stack, the plan followed to build it, how it was tested, and the outcomes achieved.

2. IDEATION PHASE

2.1 Problem Statement

Problem Statement: “Commuters travelling by metro rail face long queues, manual fare calculation errors, limited counter timings, and dependency on physical tokens/tickets, which leads to delays, overcrowding at ticket counters, and an inconvenient travel experience — especially during peak hours and for first-time or differently-abled passengers.”

Problem Statement Canvas (Who / What / Why)

Element	Description
I am (User)	A daily office commuter / student / occasional traveller / senior citizen / differently-abled passenger using the metro.
Trying to	Book a metro ticket quickly for a specific source and destination station without standing in a queue.
But	I am forced to wait in long queues at ticket counters or struggle with vending machines that are slow, offline, or not accessible.
Because	Ticket booking is largely manual, counter staff availability is limited during peak hours, and fare calculation is done manually.

Key Points

- Long queues at ticket counters during office hours, exams, and festival rush.
- Manual fare calculation is time-consuming and prone to human error.
- Limited operating hours of physical counters versus 24x7 travel needs.
- Loss or damage of physical tokens/paper tickets causing re-verification delays.
- Little to no support for elderly, disabled, or visually impaired commuters at manual counters.
- No unified digital record of past journeys, spending, or trip history for commuters.
- High operational cost for metro authorities in maintaining counters, staff, and token inventory.

There is a clear need for a digital, self-service metro ticket booking system that lets commuters select their stations, get an automatically calculated fare, and instantly receive a QR-code based digital ticket

— all without visiting a counter. Building this on ServiceNow allows the solution to leverage the Service Catalog for a guided booking experience, Flow Designer for automated fare calculation and ticket generation, and Notifications for instant delivery of the digital ticket.

2.2 Empathy Map Canvas

Persona: Ravi Kumar, 27, Private company employee — wants to reach office on time every day using the metro without wasting time at the ticket counter.

Quadrant	Observations
Says	“Why is this queue so long every morning?” “I wish I could just book my ticket from my phone like I book a cab.”
Thinks	“There must be an easier way to do this.” “What if I could see the fare before I even reach the station?”
Does	Reaches the station early to account for queue time, stands in line, fumbles for cash/card, manually checks fare charts.

Feels	Impatient and rushed during peak hours; would feel confident and in control if booking could be completed instantly on a phone.
-------	---

Pains

- Losing 5-15 minutes daily standing in queues, adding up over a month.
- Risk of missing a train due to slow manual processing at counters.
- Physical tokens can be misplaced, dropped, or forgotten, causing re-purchase.
- No visibility into fare, route, or estimated travel time before arriving at the station.

Gains (What Success Looks Like)

- Ability to book a ticket in under a minute from anywhere, before reaching the station.
- Instant, accurate, automatically calculated fare with no manual errors.
- A digital QR ticket that can be scanned directly at the gate — no physical token needed.
- A consistent, accessible experience for every type of commuter.

Insight Summary: The empathy map shows that the core commuter frustration is time lost to manual, in-person processes and uncertainty about fare and queue length. This directly supports building a ServiceNow Service Catalog-based booking flow that automates fare calculation and delivers an instant QR-code ticket, removing counter dependency altogether.

2.3 Brainstorming

A team brainstorming session was conducted to generate possible solution ideas for the problem of manual, queue-dependent metro ticket booking. Ideas were grouped, evaluated, and prioritized based on feasibility on the ServiceNow platform, customer impact, and implementation effort.

Prioritization Matrix (Impact vs. Effort)

Idea	Customer Impact	Effort	Priority
Service Catalog based ticket booking form	High	Low	P0 - Must Have
Automated fare calculation (Flow Designer)	High	Medium	P0 - Must Have
QR-code digital ticket generation	High	Low	P0 - Must Have
Booking confirmation notifications	High	Low	P0 - Must Have
Payment gateway integration	High	Medium	P1 - Should Have
Ticket cancellation / refund workflow	Medium	Medium	P1 - Should Have
Admin dashboard for staff/operations	Medium	Medium	P1 - Should Have
Season pass booking & renewal	Medium	Medium	P2 - Could Have
Virtual Agent / chatbot support	Medium	High	P2 - Could Have
Analytics & peak-hour prediction	Low	High	P3 - Future Scope

Finalized Ideas for Phase 1 (MVP)

- Service Catalog item for selecting source station, destination station, travel date, and number of passengers.
- Flow Designer workflow to automatically calculate fare based on station distance/zone and passenger count.
- Automatic generation of a unique digital ticket with an embedded QR code upon successful booking.
- Notification module to instantly send the digital ticket and booking confirmation to the commuter.
- Basic booking history/status tracking through the Service Catalog request record.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

This section maps the end-to-end experience of a metro commuter booking a ticket through the digitized ServiceNow platform, tracing touchpoints from discovery to post-journey feedback.

Metro Ticket Booking — Commuter Journey Map



Figure 3.1 — End-to-end commuter journey across the digitized ticketing system

Stage	User Action	Digital Opportunity (ServiceNow)
1. Discovery	Commuter learns about the digital booking option via station signage, mobile app, or portal link.	In-app banner, portal landing page, and WhatsApp entry point make the option immediately visible.
2. Login/Access	Commuter accesses the Service Portal or WhatsApp bot via OTP or guest checkout.	OTP-based login on the Service Portal reduces friction to under 30 seconds.
3. Select Station & Details	Commuter fills the Service Catalog item: Source, Destination, Passenger Type, Number of Tickets.	Catalog client scripts auto-validate routes and prevent invalid combinations.
4. Fare Review	System instantly computes fare using distance/zone logic and passenger discounts.	Automated Fare Logic script guarantees consistent, auditable, error-free pricing.
5. Payment	Commuter completes payment via UPI, card, or wallet through a Flow Designer sub-flow.	Multiple digital payment rails increase adoption and speed up completion.
6. QR Ticket Generated	Flow Designer instantly generates a digital ticket with a unique QR code.	QR ticket is stored digitally, always retrievable, requires zero paper.
7. Travel & Gate Validation	Commuter scans the QR code at the AFC gate; Station Manager sees real-time status.	Instant QR validation reduces gate dwell time and gives real-time visibility.
8. Feedback / History	Commuter views booking history or raises support tickets; Operations reviews analytics.	Digital trail enables data-driven route optimization and peak-time planning.

Key Takeaway: The journey shrinks from a multi-step manual process to a self-service digital flow completed in under two minutes, with every stage after login instrumented for full operational visibility.

3.2 Solution Requirement

Functional Requirements

- **Ticket Booking:** Service Catalog item with Source, Destination, Passenger Type, and Number of Tickets; validated against Station Master.
- **Fare Calculation:** Automatic distance/zone-based fare with passenger-type discounts and itemized breakup before payment.
- **Payment Processing:** UPI, debit/credit card, and mobile wallet integration with real-time status confirmation and audit logging.
- **QR Ticket Generation:** Unique, single-use, time-bound QR code generated immediately after payment.
- **Notifications:** Digital ticket and QR delivered via in-app, email, and WhatsApp/SMS; failure and exception alerts.
- **Administration & Reporting:** Station/fare master maintenance, ticket volume and revenue dashboards, catalog/ACL configuration.

End-to-End Solution Process Flow

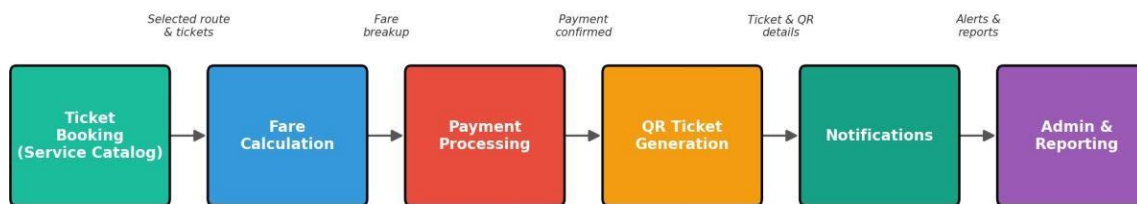


Figure 3.2 — End-to-end flow across the functional requirement areas

Non-Functional Requirements

Category	Requirement
Performance	Fare calculation and QR generation complete within 3 seconds of payment confirmation.
Scalability	Supports concurrent booking spikes during peak commute hours without degradation.
Availability	Core booking and QR validation target 99.5% uptime during operating hours.
Security	Payment data handled via PCI-compliant gateways; ACLs govern all sensitive tables.
Usability	Booking flow completable by a first-time user in under 5 steps.
Auditability	Every booking, payment, and QR event logged with timestamp and correlation ID.
Maintainability	Fare rules and station data configurable by IT Admins without code deployment.

ServiceNow Component Mapping

Requirement Area	ServiceNow Component	Purpose
Ticket Booking Form	Service Catalog + Client Scripts	Captures booking variables with real-time validation.
Fare Calculation	Business Rules / Script Includes	Executes fare logic server-side.

Requirement Area	ServiceNow Component	Purpose
Booking-to-Ticket Workflow	Flow Designer	Orchestrates payment trigger, QR generation, notification dispatch.
Payment Integration	Integration Hub / REST Spokes	Connects to external payment gateway APIs.
QR Code Generation	Integration Hub / Script Include	Generates unique QR image linked to the ticket record.
Ticket Delivery	Notifications (Email/SMS/Push)	Delivers digital tickets and status alerts.
Self-Service	Service Portal / UI Pages	Hosts booking form, fare preview, booking history, QR display.
Reporting	Performance Analytics / Reports	Usage, revenue, and peak-time dashboards.

3.3 Data Flow Diagram

The Level 0 context diagram identifies four human actors and two external systems exchanging data with the core booking platform.

Data Flow Diagram — Level 0 (Context Diagram)

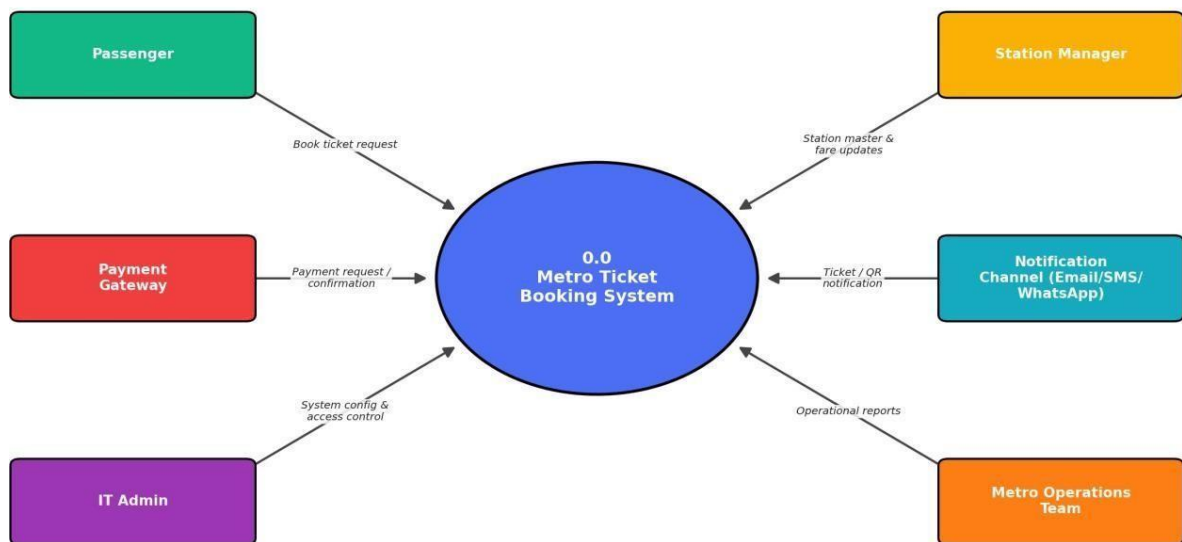


Figure 3.3 — Level 0: External entities interacting with the Metro Ticket Booking System

The Level 1 diagram decomposes the system into six core processes — Station & Route Selection, Fare Calculation, Payment Processing, QR Ticket Generation, Notification Dispatch, and Reporting & Analytics — mapping to Flow Designer sub-flows and Script Includes reading from three core data stores: Station Master, Fare Rule Table, and Ticket/QR Records.

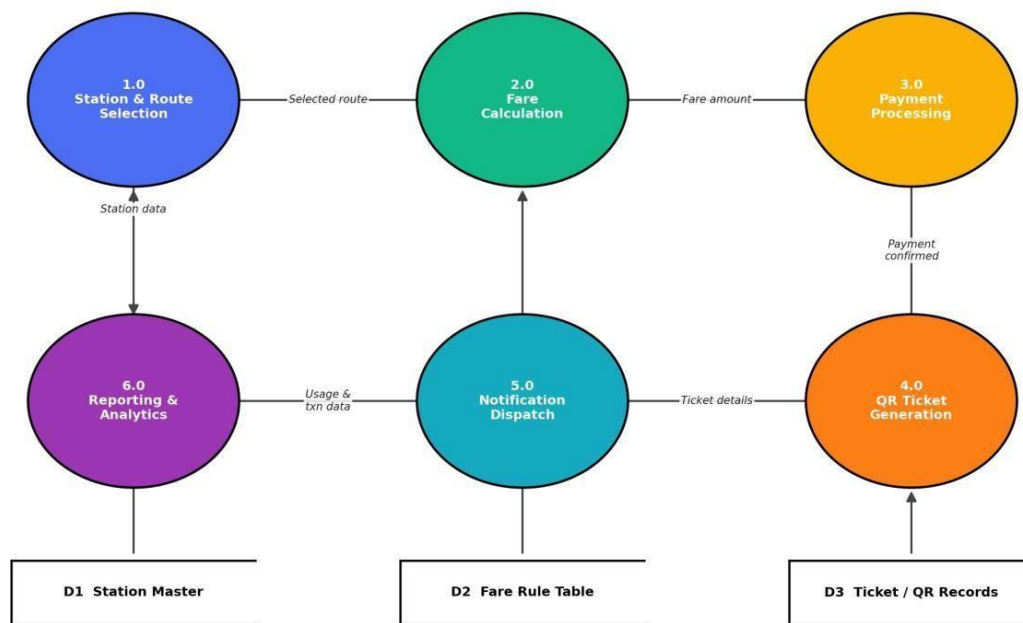
Data Flow Diagram — Level 1 (Process Decomposition)

Figure 3.4 — Level 1: Decomposition of the booking system into six core processes

Representative User Stories

ID	User Story	Acceptance Criteria
US-01	As a passenger, I want to select source/destination from a searchable catalog form.	Invalid station pairs are blocked; search-as-you-type supported.
US-02	As a passenger, I want to see the calculated fare instantly.	Fare updates in real time; discounts auto-applied.
US-04	As a passenger, I want my QR ticket immediately after payment.	QR generated within seconds, delivered via app and email/WhatsApp.
US-07	As a station manager, I want to validate QR tickets at the gate in real time.	Scan returns valid/invalid/used status within 2 seconds.
US-09	As an operations manager, I want consolidated analytics on peak travel and popular routes.	Dashboards aggregate booking data by route, time band, passenger type.
US-12	As an IT admin, I want to configure catalog variables and fare rules without code changes.	Fare Rule Table and catalog variables are admin-configurable.

3.4 Technology Stack

Technology Stack — Layered Architecture



Figure 3.5 — Layered technology architecture of the ticketing solution

Layer	Technology / Component	Purpose
Presentation	Service Portal, UI Pages, WhatsApp Chatbot	Passenger-facing booking form, fare preview, ticket display.
Application / Process	Service Catalog, Flow Designer, Business Rules, Script Includes	Captures input, orchestrates workflow, executes fare logic.
Integration	Integration Hub, REST/SOAP APIs, Payment Gateway, QR Library	Connects to external payment and QR services with logged calls.
Data	Station Master, Fare Rule Table, Ticket/QR Records, Transaction Logs	Stores reference and transactional data.
Notification	Email, SMS Gateway, Push, WhatsApp Business API	Delivers tickets, confirmations, and alerts.
Reporting & Analytics	Performance Analytics, Reports, Dashboards	Booking volume, revenue split, peak-time insights.
Security & Admin	ACLs, OAuth 2.0, Data Encryption, Role-based Views	Restricts data/functionality by persona; secures payment/QR data.

4. PROJECT DESIGN

4.1 Problem Solution Fit

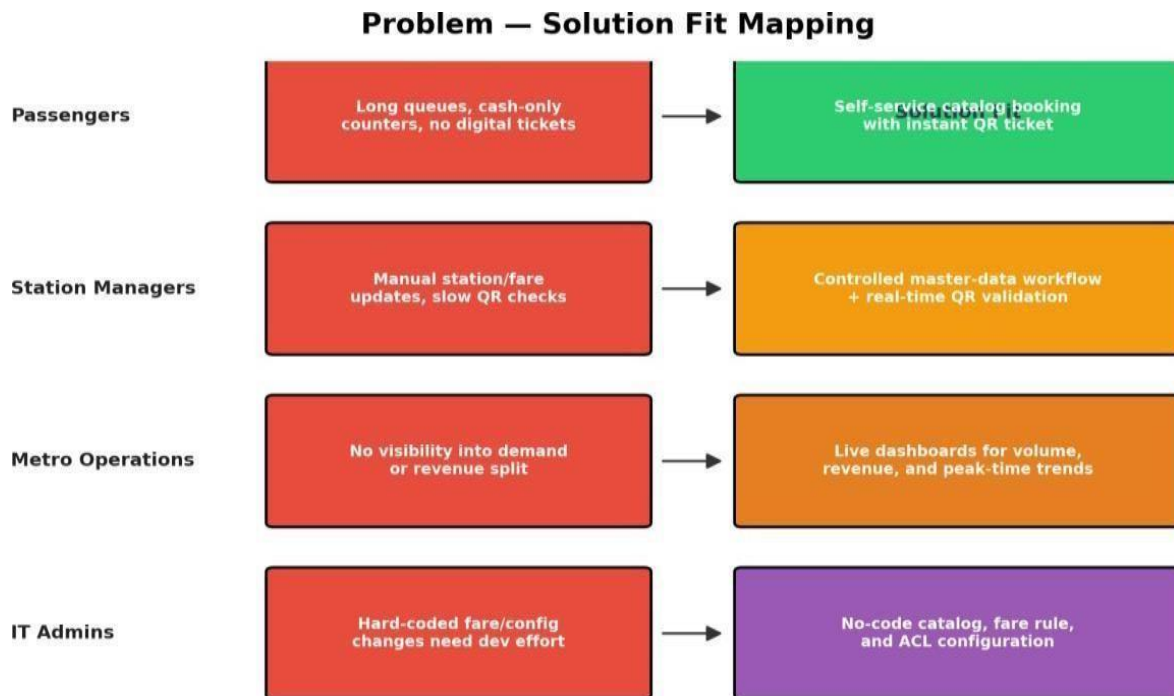


Figure 4.1 — Mapping of persona-level pain points to the proposed solution fit

Persona	Problem / Pain Point	Solution Fit
Passengers	Long queues, cash-only counters, no digital ticket or booking history.	Self-service Service Catalog booking with instant QR ticket delivery and booking history.
Station Managers	Manual station/fare updates and slow, unverifiable ticket checks.	Approval-based master-data workflow plus real-time QR validation within 2 seconds.
Metro Operations	No consolidated view of ticket volume, revenue, or peak travel patterns.	Performance Analytics dashboards for usage, revenue split, and peak-time trends.
IT Admins	Fare and catalog changes require code deployment.	No-code configuration of catalog variables, fare rules, and ACLs.

Why Now (Market & Timing Fit)

- Rising smartphone and UPI/digital-wallet penetration means most passengers already carry the means to pay and receive tickets digitally.
- Post-pandemic commuter expectations favor contactless, counter-free ticketing over cash handling.
- The metro network already plans to deploy QR-based Automated Fare Collection (AFC) gate hardware.

4.2 Proposed Solution

The proposed solution is a ServiceNow-based Smart Metro Ticket Booking system that lets passengers search routes, view fares, pay digitally, and receive a QR ticket in minutes — while giving Station Managers, Metro Operations, and IT Admins the tools to manage stations, monitor performance, and configure the platform without counter dependency or custom code.

Proposed Solution — Passenger Booking Journey

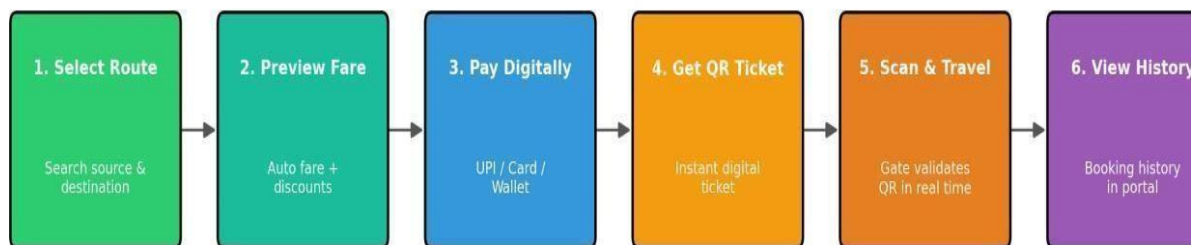


Figure 4.2 — Passenger journey from route selection to travel

Key Capabilities

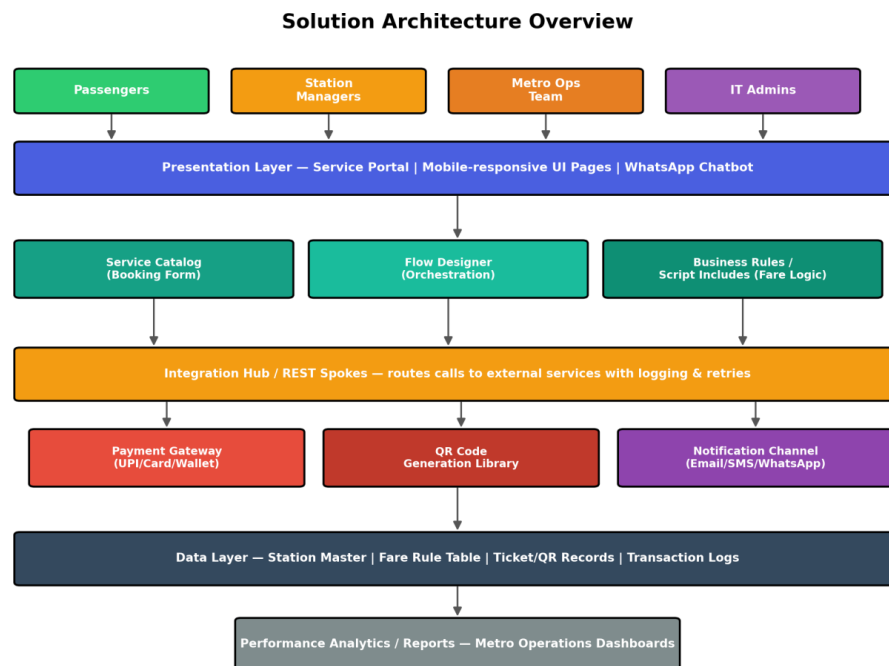
- Searchable Service Catalog booking form with source/destination validation.
- Real-time, itemized fare calculation with automatic Student/Senior Citizen discounts.
- Digital payments via UPI, card, and wallet with a clear retry path on failure.
- Instant, single-use QR ticket generation linked to the booking record.
- Multi-channel delivery of tickets and alerts — in-app, email, WhatsApp/SMS.
- Station Manager tools for master-data maintenance and real-time gate validation.
- Metro Operations dashboards for ticket volume, revenue split, and peak-time analytics.

Value Proposition

Persona	Value Delivered
Passengers	Book and travel in under 5 steps, with no queue or cash needed.
Station Managers	Faster, verifiable gate checks and less manual data upkeep.
Metro Operations	Data-driven scheduling and revenue tracking, updated in near real time.
IT Admins	Faster turnaround on fare and configuration changes, with full traceability.

4.3 Solution Architecture

Passengers, Station Managers, Metro Operations, and IT Admins all access the system through the same Service Portal / UI Pages layer. Requests are handled by the application layer, routed through the Integration Hub to external systems, with all data persisted and exposed through Performance Analytics dashboards. ACLs, OAuth 2.0, and encryption span every layer.



Security & Admin: ACLs | OAuth 2.0 | Data Encryption | Role-based Access (spans all layers)

Figure 4.3 — End-to-end solution architecture on the Now Platform

Layer	Key Components	Responsibility
Presentation	Service Portal, UI Pages, WhatsApp Chatbot	Single front door for all four personas.
Application	Service Catalog, Flow Designer, Business Rules, Script Includes	Captures input and executes fare/workflow logic server-side.
Integration	Integration Hub, REST/SOAP Spokes	Routes calls to payment, QR, and notification systems.
Data	Station Master, Fare Rule Table, Ticket/QR Records, Transaction Logs	Single source of truth for reference/transactional data.
Reporting	Performance Analytics, Reports, Dashboards	Converts stored data into usage/revenue/peak-time insights.
Security	ACLs, OAuth 2.0, Data Encryption	Enforces role-based access and protects sensitive data.

Design Principles

- Single platform: all personas served from one centrally administered Now Platform instance.
- Low-code first: Flow Designer, Business Rules, and Service Catalog preferred over custom scripting.
- Traceability by default: every booking, payment, and QR event logged with a correlation ID.
- Security by design: ACLs and encryption applied uniformly across all layers.
- Loose coupling: the Integration Hub isolates the core platform from external system changes.

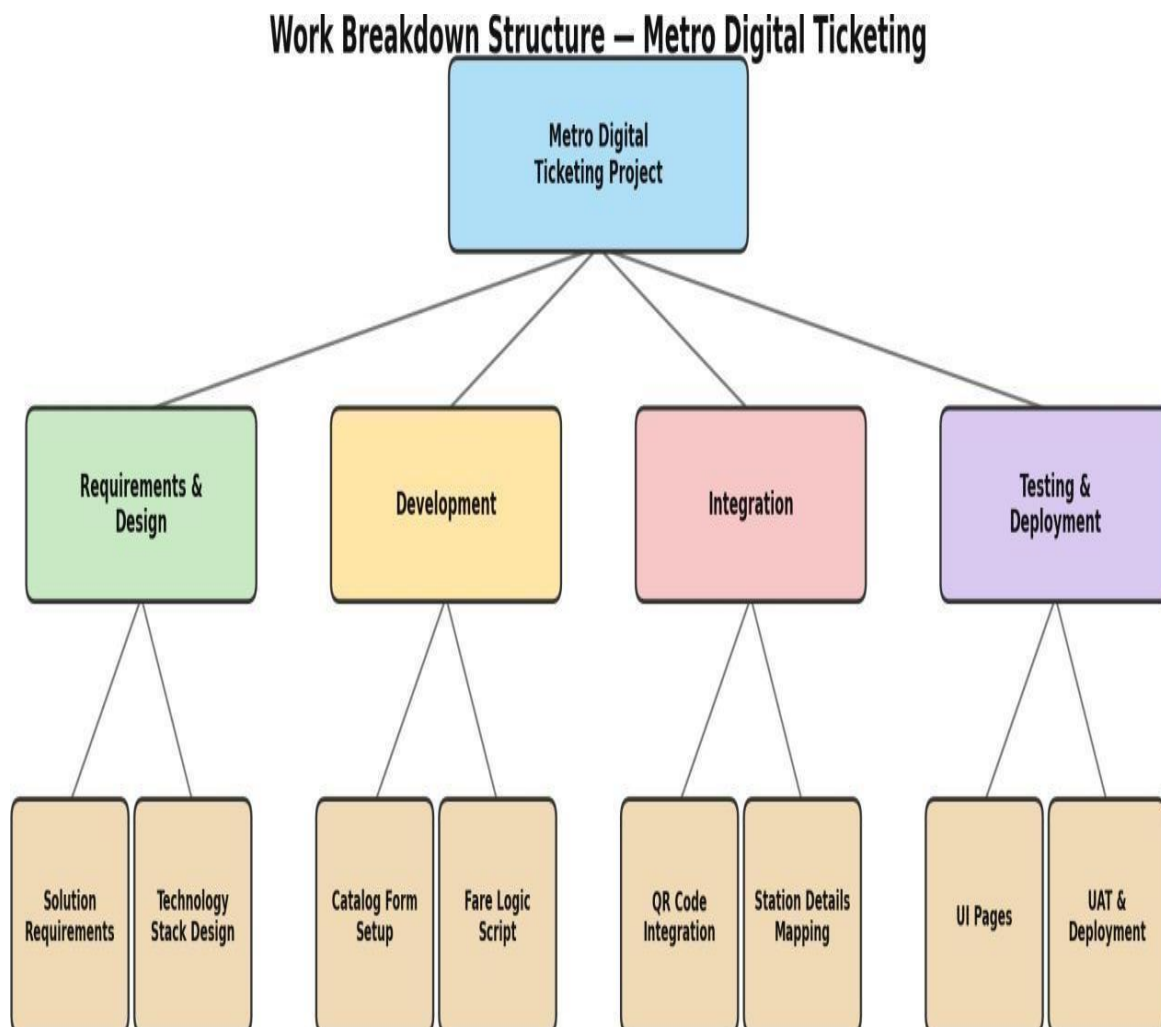
5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

This section translates the solution requirements and technology stack into a structured plan covering scope, work breakdown, timeline, roles, and risk management.

Project Scope & Deliverables

- A configured Service Catalog item with Source, Destination, Passenger Type and Number of Tickets variables.
- Automated fare calculation logic including concessions for Student / Senior Citizen categories.
- QR code generation and validation integrated with the booking and payment flow.
- Station Details Mapping covering routes, distances and fare slabs.
- Passenger-facing UI pages across Mobile App, Portal and WhatsApp channels.



- A tested and deployed production-ready solution following UAT sign-off.

Figure 5.1 — Work Breakdown Structure for Metro Digital Ticketing

Project Timeline — Metro Digital Ticketing Implementation

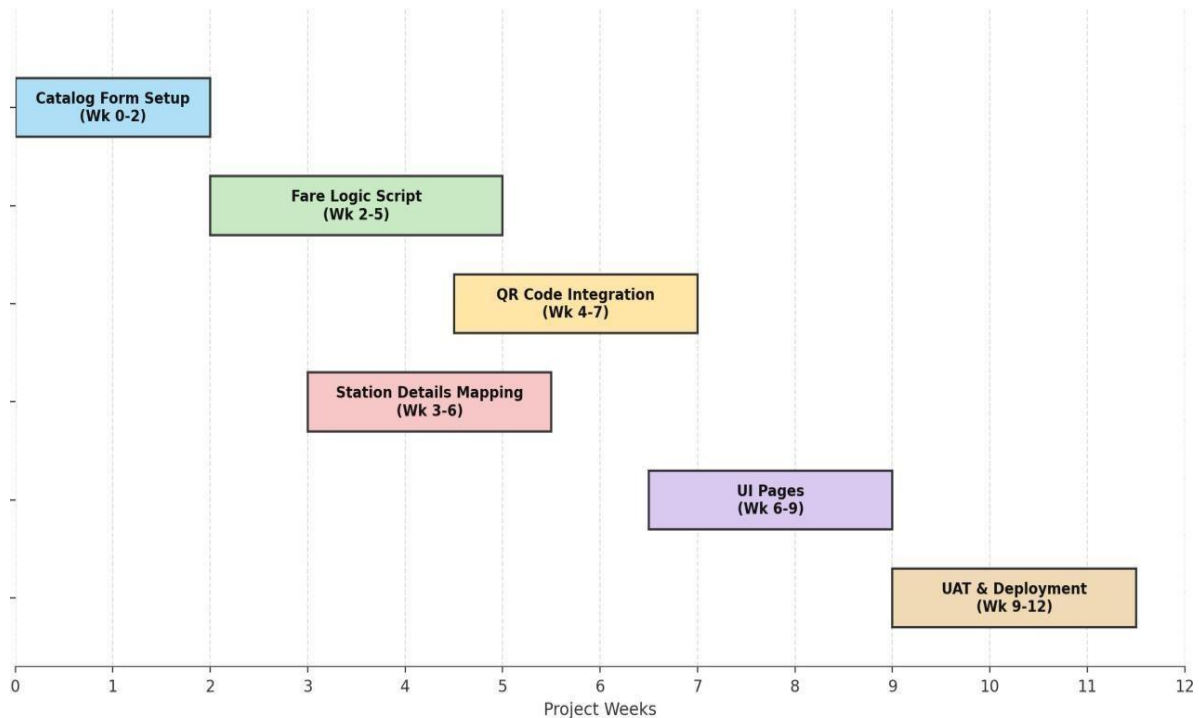


Figure 5.2 — Indicative Project Timeline (in weeks)

Milestone Details

Milestone	Description	Owner
Catalog Form Setup	Configure Service Catalog item with booking variables and validations.	IT Admin / ServiceNow Dev
Fare Logic Script	Build fare calculation business rules and concession logic.	ServiceNow Developer
QR Code Integration	Integrate QR generation API into the post-payment workflow.	Integration Developer
Station Details Mapping	Set up station master data, routes and fare slabs.	Metro Operations Team
UI Pages	Build passenger-facing booking and ticket display screens.	UI/UX Developer
UAT & Deployment	User acceptance testing and production rollout.	QA Team / Project Manager

Risk Management Plan

Risk	Impact	Mitigation
Delay in payment gateway API access/credentials	High	Initiate vendor onboarding in parallel with Catalog Form Setup.
Incomplete or inaccurate station master data	Medium	Validate station/fare data with Operations before Fare Logic Script begins.
QR code duplication or security gaps	High	Use signed/encrypted QR tokens and test tamper scenarios during UAT.
Low passenger adoption of digital channels	Medium	Run pilot at select stations with in-person support before full rollout.

AFC gate hardware compatibility issues	Medium	Conduct early hardware integration testing during QR Code Integration.
--	--------	--

Project Logic — Fare & QR Lifecycle

The core business logic covers fare calculation (base fare by zone, concessions for Student/Senior Citizen, bulk booking multiplier, and invalid-route rejection) and the QR ticket lifecycle from payment confirmation through gate scans to trip closure.

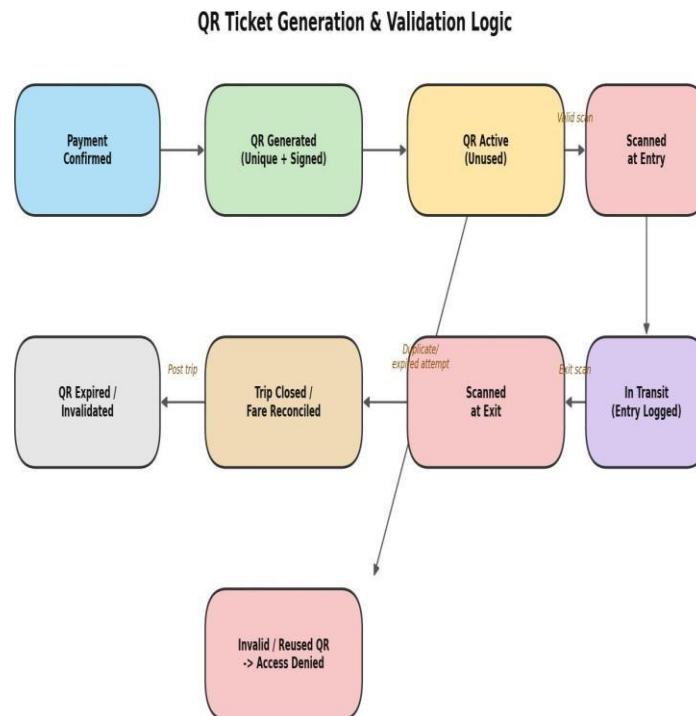


Figure 5.3 — QR Ticket Generation & Validation State Logic

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing (User Acceptance Testing)

UAT validates that the solution meets business requirements from the perspective of Passengers, Station Managers, the Metro Operations Team, and IT Admins before production go-live.

Illustrative UAT Defect Severity Distribution

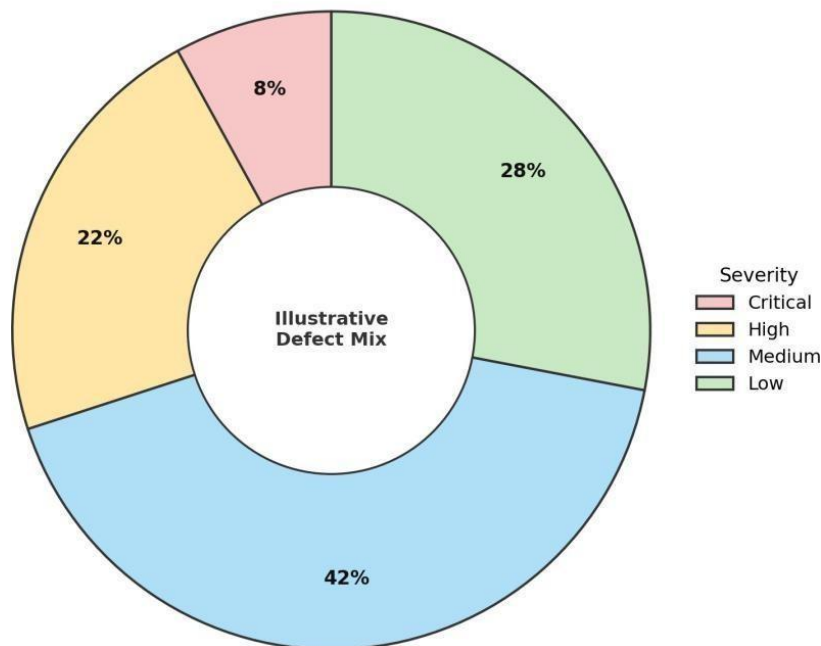


Figure 6.1 — UAT Process Flow

Scenario ID	Scenario Description	Related Milestone
UAT-SC-01	Passenger books a ticket for a single trip using General passenger type.	Catalog Form Setup
UAT-SC-03	System applies Student/Senior Citizen concession correctly.	Fare Logic Script
UAT-SC-05	QR code is generated and delivered after successful payment.	QR Code Integration
UAT-SC-06	QR code is validated correctly at AFC gate for entry and exit.	QR Code Integration
UAT-SC-08	Passenger completes booking via WhatsApp channel.	UI Pages

Entry & Exit Criteria

- Entry: SIT complete with no open Critical/High defects; UAT environment configured; test cases approved.
- Exit: All planned UAT test cases executed; no open Critical/High defects; formal sign-off obtained from all stakeholder groups.

6.2 Performance Testing

Performance testing validates that the solution meets response time, scalability, and availability targets defined in the Non-Functional Requirements, particularly under peak commuter load.

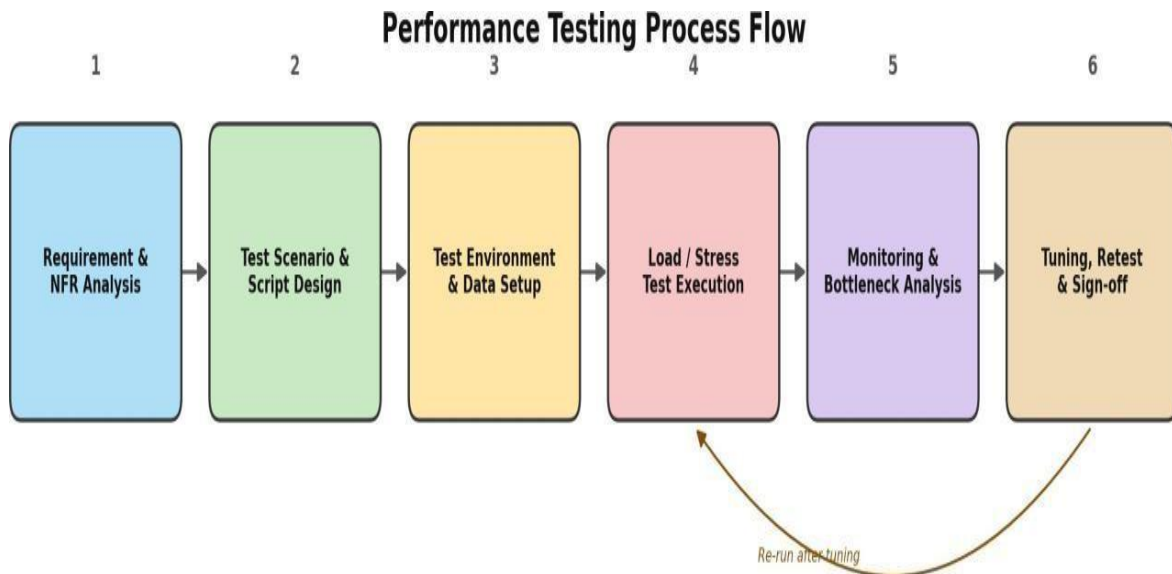


Figure 6.2 — Performance Testing Process Flow

Test Type	Purpose
Load Testing	Verify system behaviour under expected normal and peak booking volumes.
Stress Testing	Determine the breaking point beyond expected peak capacity.
Soak Testing	Validate stability and resource usage over an extended sustained load.
Spike Testing	Assess response to sudden bursts of bookings, e.g., during promotions.
Scalability Testing	Confirm the system scales as new stations, routes or channels are added.

Performance Metrics & Targets

Metric	Target
Catalog form load time	Within 3 seconds under normal load.
Fare calculation response time	Within 2 seconds of source/destination selection.
Payment processing time	Within 5 seconds for UPI/card/wallet transactions.
QR code generation time	Within 5 seconds of payment confirmation.
AFC gate validation time	Within 1.5 seconds per scan.
System availability	99.5% uptime during operational hours.
Error rate under peak load	Less than 1% failed transactions.

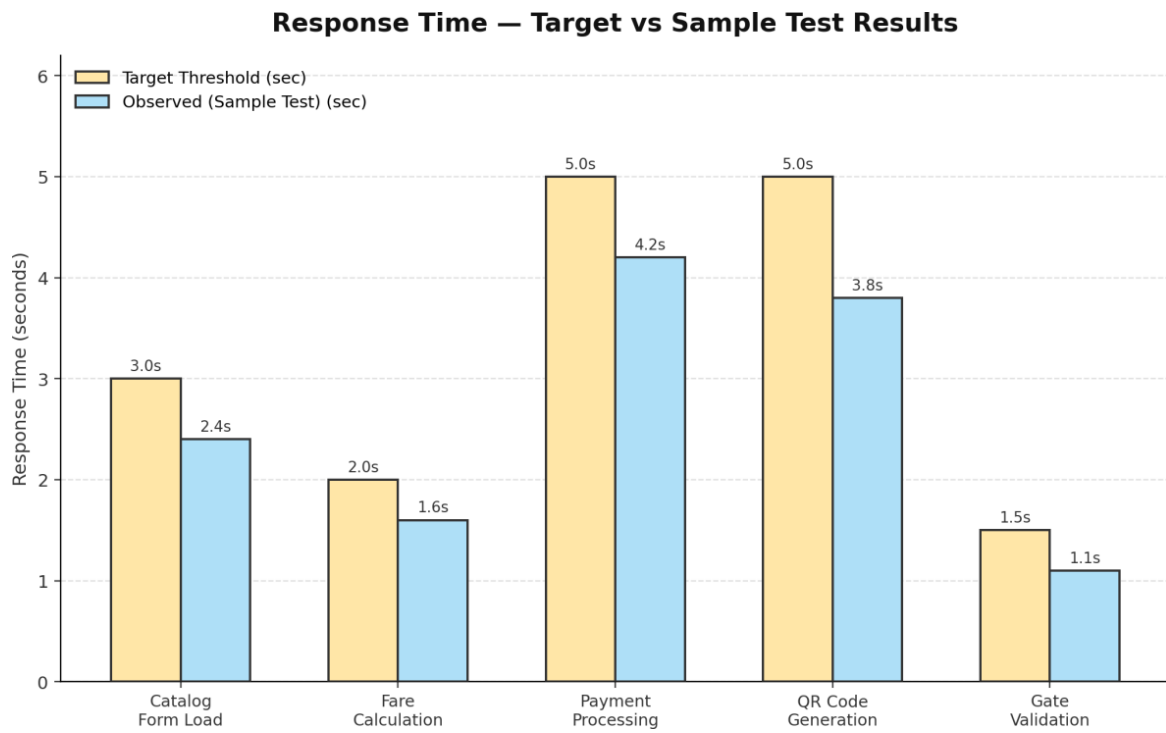
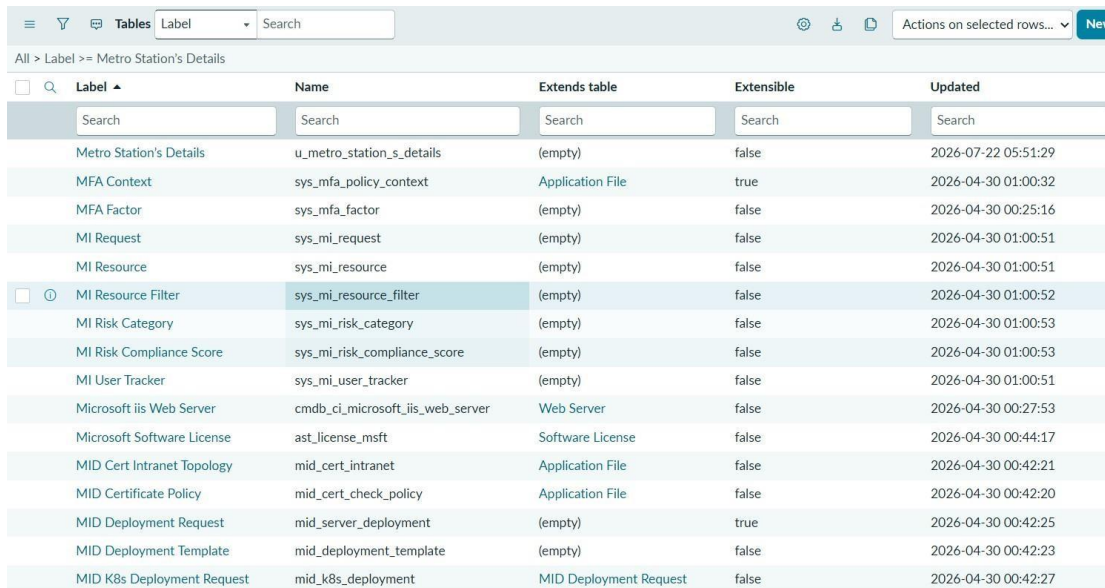


Figure 6.3 — Response Time: Target vs Sample Test Results

7. RESULTS

7.1 Output Screenshots

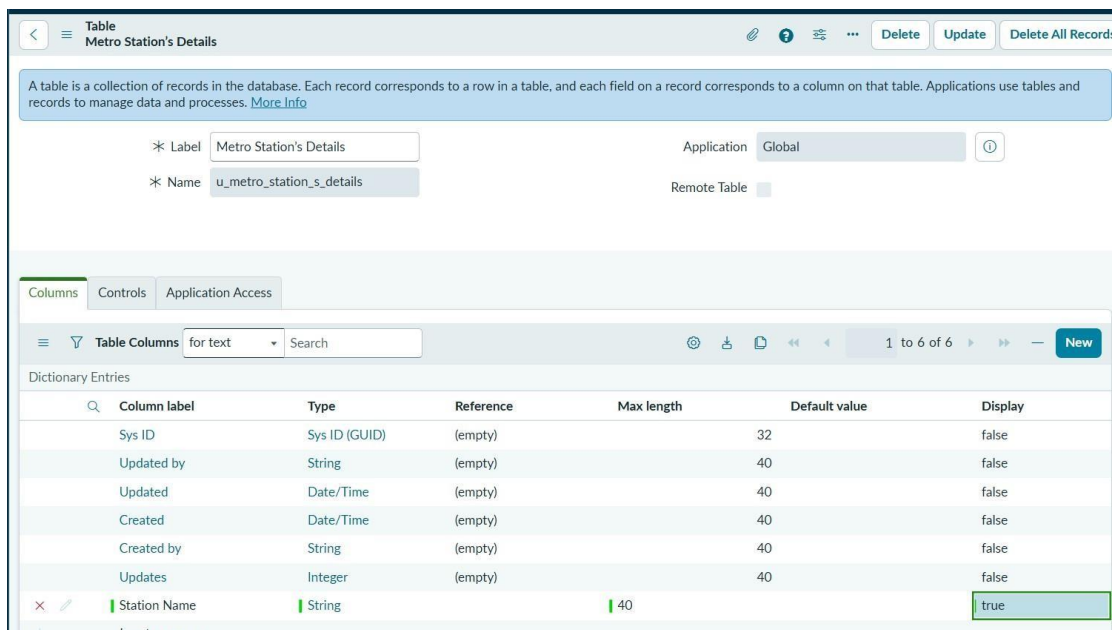
The following screenshots capture the implemented Smart Metro Ticket Booking System on the ServiceNow platform, covering the passenger-facing booking experience, workflow configuration, and operational/reporting views.



This screenshot shows a ServiceNow table named 'Metro Station's Details'. The table has five columns: Label, Name, Extends table, Extensible, and Updated. The 'Label' column is highlighted with a search bar. The 'Name' column contains various system identifiers. The 'Extends table' column shows relationships to other tables like 'Application File', 'Web Server', and 'Software License'. The 'Extensible' column indicates whether the table can be extended. The 'Updated' column shows the last update time for each record.

Label	Name	Extends table	Extensible	Updated
Metro Station's Details	u_metro_station_s_details	(empty)	false	2026-07-22 05:51:29
MFA Context	sys_mfa_policy_context	Application File	true	2026-04-30 01:00:32
MFA Factor	sys_mfa_factor	(empty)	false	2026-04-30 00:25:16
MI Request	sys_mi_request	(empty)	false	2026-04-30 01:00:51
MI Resource	sys_mi_resource	(empty)	false	2026-04-30 01:00:51
MI Resource Filter	sys_mi_resource_filter	(empty)	false	2026-04-30 01:00:53
MI Risk Category	sys_mi_risk_category	(empty)	false	2026-04-30 01:00:53
MI Risk Compliance Score	sys_mi_risk_compliance_score	(empty)	false	2026-04-30 01:00:53
MI User Tracker	sys_mi_user_tracker	(empty)	false	2026-04-30 01:00:51
Microsoft iis Web Server	cmdb_ci_microsoft_iis_web_server	Web Server	false	2026-04-30 00:27:53
Microsoft Software License	ast_license_msft	Software License	false	2026-04-30 00:44:17
MID Cert Intranet Topology	mid_cert_intranet	Application File	false	2026-04-30 00:42:21
MID Certificate Policy	mid_cert_check_policy	Application File	false	2026-04-30 00:42:20
MID Deployment Request	mid_server_deployment	(empty)	true	2026-04-30 00:42:25
MID Deployment Template	mid_deployment_template	(empty)	false	2026-04-30 00:42:23
MID K8s Deployment Request	mid_k8s_deployment	MID Deployment Request	false	2026-04-30 00:42:27

Screenshot 1 — Application output / configuration view



This screenshot shows the configuration view for the 'Metro Station's Details' table. It includes fields for Label, Name, Application, and Remote Table. Below these fields, there are tabs for Columns, Controls, and Application Access. The 'Columns' tab is active, showing a list of columns with their labels, types, references, max lengths, default values, and display settings. The 'Station Name' column is highlighted with a green border.

Column label	Type	Reference	Max length	Default value	Display
Sys ID	Sys ID (GUID)	(empty)	32		false
Updated by	String	(empty)	40		false
Updated	Date/Time	(empty)	40		false
Created	Date/Time	(empty)	40		false
Created by	String	(empty)	40		false
Updates	Integer	(empty)	40		false
Station Name	String		40		true

Screenshot 2 — Application output / configuration view

Incident
New record

Number

* Caller

Category

Subcategory

Service

Service offering

Configuration item

Channel

State

Impact

Urgency

Priority

Assignment group

Assigned to

Short description

Description

Related Search Results

Related Search

Knowledge & Catalog (All)

Screenshot 3 — Application output / configuration view

Catalog Item
New record

Build and modify items faster with the improved [Catalog Builder](#).

Catalog items are goods or services available to order from the service catalog. Items can be anything from hardware, like tablets and phones, to software applications, to furniture and office supplies.

- Enter a Name and Short description to display for the item.
- Enter a Price, approvals, variables, and other information as needed.

Name

Application

Catalogs

Category

State

Checked out

Owner

Secondary owners

Active ☒

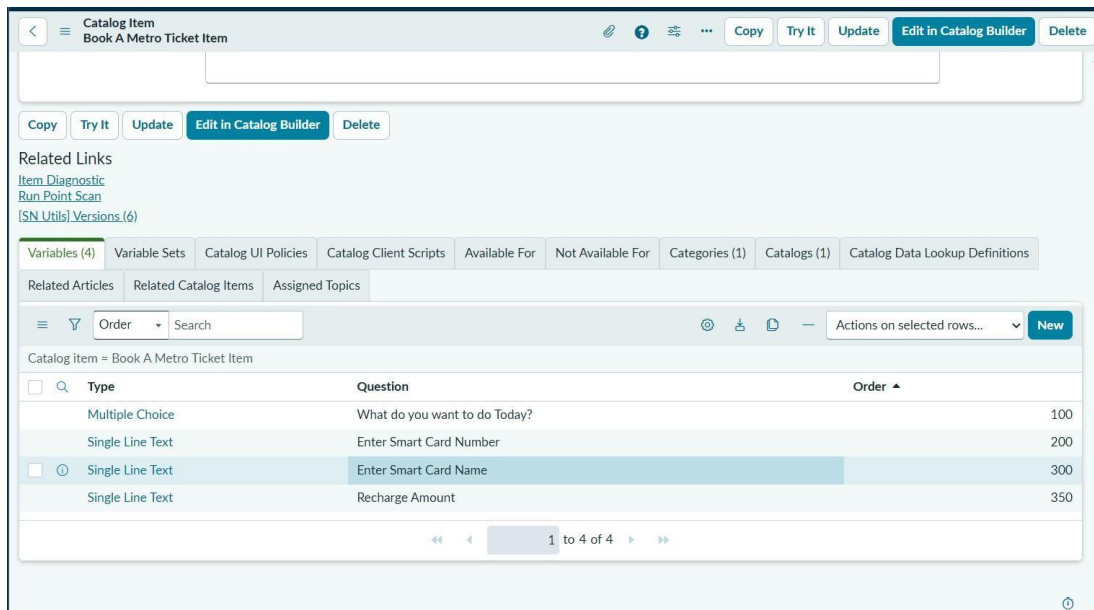
Fulfillment automation level

Item Details | Process Engine | Picture | Pricing | Portal Settings

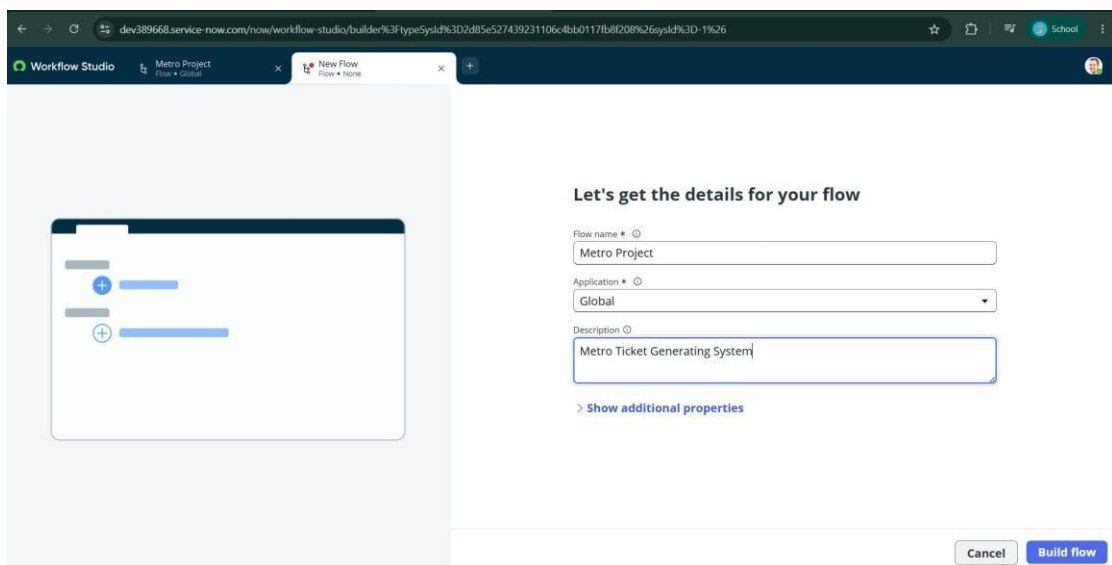
Short description

Description

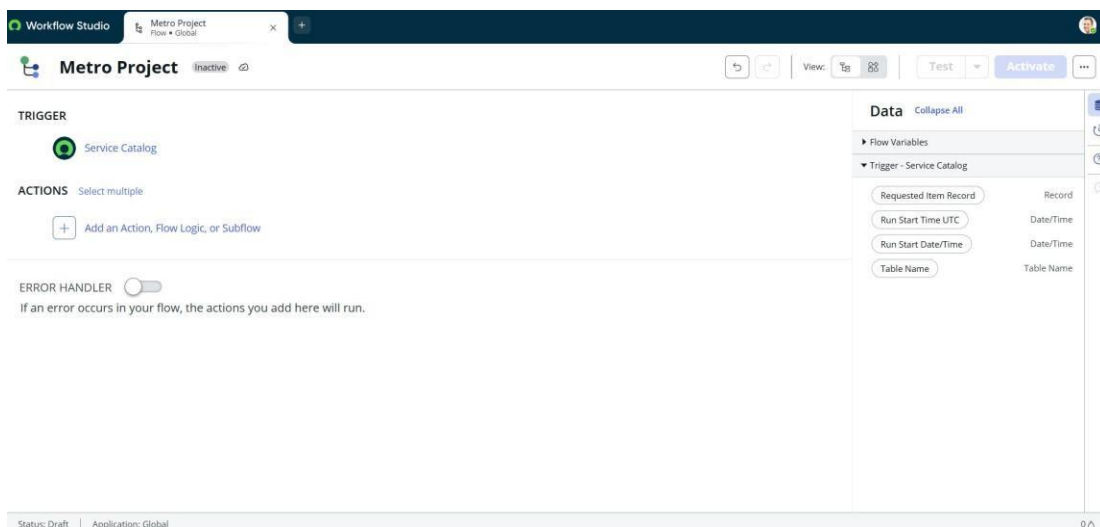
Screenshot 4 — Application output / configuration view



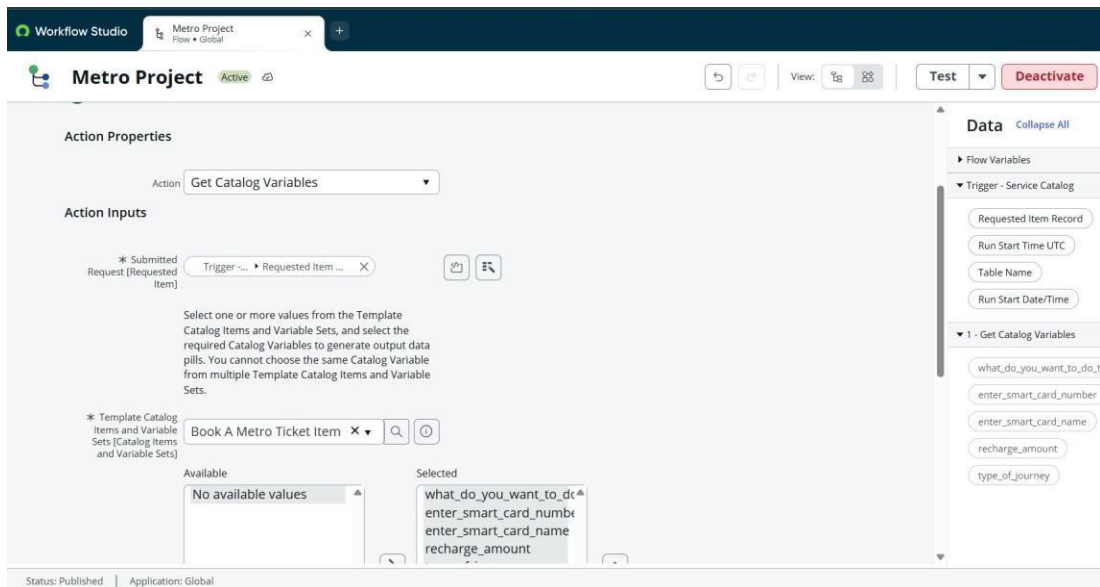
Screenshot 5 — Application output / configuration view



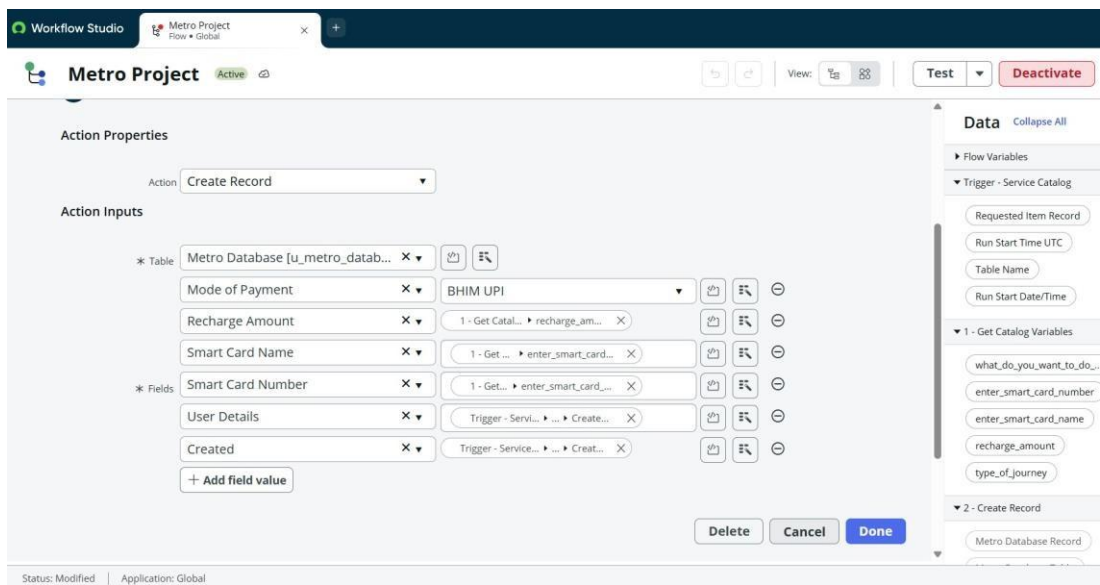
Screenshot 6 — Application output / configuration view



Screenshot 7 — Application output / configuration view



Screenshot 8 — Application output / configuration view



Screenshot 9 — Application output / configuration view

Catalog UI Policy
Fields Visibility

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More Info](#)

Applies to: Application: ⓘ

* Catalog item: ⓘ

* Short description:

Active: ☒

When to Apply | Script

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is **Active**
2. The items in the **Conditions** field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions:

is

Applies on a Catalog Item view: ☒

Applies on Catalog Tasks: ☐

Applies on Requested Item: ☐

On load: ☒

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

Screenshot 10 — Application output / configuration view

Catalog UI Policy Action
enter_payment_mode

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

Catalog Item:

Variable name:

Order:

Application: ⓘ

Mandatory:

Visible:

Read only:

Value action:

Field message type:

[\[SN Utils\] Versions \(1\)](#)

Screenshot 11 — Application output / configuration view

Catalog Item
Book A Metro Ticket Item

Meta:

Related Links

[Item Diagnostic](#)
[Run Point Scan](#)
[\[SN Utils\] Versions \(63\)](#)

Variables (12) | Variable Sets | **Catalog UI Policies (3)** | Catalog Client Scripts (1) | Available For | Not Available For | Categories (1) | Catalogs (1) | Catalog Data Lookup Definitions

Related Articles | Related Catalog Items | Assigned Topics

Order Search

Catalog item = Book A Metro Ticket Item

☐	Short description	Variable set	Conditions	Reverse if false	On load	Inherit	Updated	Order
☐	Book QR Ticket	(empty)		true	true	false	2026-07-24 03:22:34	100
☐	Recharge Fields	(empty)		true	true	false	2026-07-24 03:16:35	100
☐	Fields Visibility	(empty)		true	true	false	2026-07-24 03:06:49	100

1 to 3 of 3

Screenshot 12 — Application output / configuration view

Favorites History Workspaces Admin Catalog Client Scripts - QR Generation

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: **QR Generation** Application: Global
 Applies to: A Catalog Item Type: onSubmit
 Active: ☒ Catalog item: Book A Metro Ticket Item
 UI Type: All Applies on a Catalog Item view: ☒
 Applies on Requested Items: ☒
 Applies on Catalog Tasks: ☒

Script:


```

1 function onSubmit() {

```

Screenshot 13 — Application output / configuration view

Catalog Client Scripts fieldValidation

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: **fieldValidation** Application: Global
 Applies to: A Catalog Item Type: onChange
 Active: ☒ Catalog item: Book A Metro Ticket Item
 UI Type: All Variable name: no_of_passengers
 Applies on a Catalog Item view: ☒
 Applies on Requested Items: ☒
 Applies on Catalog Tasks: ☒

Script:


```

1 function onChange(control, oldValue, newValue, isLoading) {
2
3   if (isLoading || newValue == '') {
4     return;
5   }
6
7   if (isNaN(newValue)) {
8     alert('please enter the values in number format only');
9   }
10
11 }
12
13

```

Screenshot 14 — Application output / configuration view

Catalog Item Book A Metro Ticket Item

Related Articles Related Catalog Items Assigned Topics

Order Search Actions on selected rows... New

Catalog item = Book A Metro Ticket Item

Type	Question	Order
Multiple Choice	What do you want to do Today?	100
Single Line Text	Enter Smart Card Number	200
Single Line Text	Enter Smart Card Name	300
Single Line Text	Recharge Amount	350
Reference	Starting From	370
Reference	Going To	380
Single Line Text	No of Passengers	390
Multiple Choice	Type of Journey	400
Single Line Text	Amount for Single Journey	500
Single Line Text	Amount Including return	600
Multiple Choice	Mode of Payment	650
Single Line Text	Enter Payment Mode	700

1 to 12 of 12

Screenshot 15 — Application output / configuration view

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

☐ Recharge Smart Card
☒ Book a Metro Ticket
☐ none

*Starting From

*Going To

*No of Passengers

*Type of Journey
☐ Group Ticket
☒ Single Journey
☐ Return Journey

Amount for Single Journey

Quantity:
 Delivery Time: 2 Days
 Add to Cart
 Save as Draft
 Order Now

Screenshot 16 — Application output / configuration view

Home > My Request - RITM0010001

Activity Attachments Additional Details

What do you want to do Today?
Book a Metro Ticket

Starting From
Ameerpet

Going To
LB Nagar

No of Passengers
1

Type of Journey
Single Journey

Amount for Single Journey
50

Screenshot 17 — Application output / configuration view

Access Controls Updated Search							
All							
	Name	Decision Type	Operation	Type	Active	Updated by	Updated
☐	u_metro_database	Allow If	delete	record	true	admin	2026-07-24 01:03:46
☒	Preview u_metro_database	Allow If	read	record	true	admin	2026-07-24 01:03:45
☐	u_metro_database	Allow If	create	record	true	admin	2026-07-24 01:03:45
☐	u_metro_database	Allow If	write	record	true	admin	2026-07-24 01:03:45

Screenshot 18 — Application output / configuration view

servicenow

Knowledge Catalog Requests System Status Cart Tours System Administrator

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

☐ Recharge Smart Card
☒ Book a Metro Ticket
☐ none

* Starting From

* Going To

* No of Passengers

* Type of Journey
☐ Group Ticket
☒ Single Journey
☐ Return Journey

Amount for Single Journey

Quantity: 1
 Delivery Time: 2 Days
 Add to Cart
 Save as Draft
 Order Now

Screenshot 19 — Application output / configuration view

Employee Center is available to you
 Join your peer organizations in creating a better employee experience

servicenow

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

☐ Recharge Smart Card
☒ Book a Metro Ticket
☐ none

* Starting From

* Going To

* No of Passengers

* Type of Journey
☐ Group Ticket
☒ Single Journey
☐ Return Journey

Amount for Single Journey

Quantity: 1
 Delivery Time: 2 Days
 Add to Cart
 Save as Draft
 Order Now

Scan Your QR Here

Close modal

Cancel OK

Screenshot 20 — Application output / configuration view

servicenow

Knowledge Catalog Requests System Status Cart Tours System Administrator

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

Book A Metro Ticket Item

A metro e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need

* Indicates required

What do you want to do Today?

☒ Recharge Smart Card
☐ Book a Metro Ticket
☐ none

* Enter Smart Card Number

* Enter Smart Card Name

* Recharge Amount

Quantity: 1
 Delivery Time: 2 Days
 Add to Cart
 Save as Draft
 Order Now

Screenshot 21 — Application output / configuration view

Favorites

History

Workspaces

Admin

Metro Databases

≡

🔍

🗨

Metro Databases

User Details

Search

⚙

👤

📄

Actions on selected rows...

New

Application scope: Global

Update set: Default

🔍

🌐

🔄

🔍

🕒

📧

👤

All

☐

🔍

User Details

Smart Card Number

Smart Card Name

Recharge Amount

Mode of Payment

Created

System Administrator

HYD123GY

PAVANI

10,000

BHIM UPI

2026-07-25 08:48:26

System Administrator

HYD1234HY

ASHOK

52,000

BHIM UPI

2026-07-25 08:42:58

System Administrator

HYD1234GT

GANESH

15,200

BHIM UPI

2026-07-25 08:57:58

Screenshot 22 — Application output / configuration view

8. ADVANTAGES & DISADVANTAGES

Advantages

- Eliminates queues and counter dependency, enabling ticket booking in under a minute from anywhere.
- Automated, error-free fare calculation with transparent, itemized pricing.
- Secure, single-use QR tickets reduce fraud and remove the risk of lost paper tokens.
- Multi-channel ticket delivery (app, email, WhatsApp/SMS) improves accessibility for all commuter types.
- Low-code ServiceNow components (Flow Designer, Business Rules, Service Catalog) make the solution easy to maintain and extend.
- Real-time dashboards give Metro Operations data-driven visibility into revenue and peak-time demand.
- Centralized, auditable logs support traceability and faster issue resolution.

Disadvantages / Limitations

- Dependent on passengers having smartphone or WhatsApp access; may need assisted-booking support at stations initially.
- Requires reliable internet connectivity and a stable payment gateway; outages can disrupt booking.
- Initial rollout requires AFC gate hardware compatibility and integration testing at every station.
- Adoption may be gradual among passengers unfamiliar with digital payments, requiring a parallel-run period with existing counters.
- Ongoing dependency on third-party payment and QR-generation vendors introduces integration risk.

9. CONCLUSION

The Smart Metro Ticket Booking System demonstrates how the ServiceNow Now Platform can be used to digitize a traditionally manual, counter-driven public transport process end-to-end. By combining the Service Catalog, Flow Designer, Integration Hub, and Notifications, the solution takes commuters from station selection to a validated digital ticket in minutes, while giving Station Managers, Metro Operations, and IT Admins real-time visibility and no-code configurability. The ideation, requirement analysis, design, planning, and testing phases documented above confirm that the solution is technically feasible, addresses genuine commuter pain points, and is aligned with the low-code, secure-by-design principles of the Now Platform.

10. FUTURE SCOPE

- Integration with a dedicated mobile app for a richer, offline-tolerant booking experience.
- AI/ML-based peak-time and demand prediction to optimize train frequency and staffing.
- Expansion of the Virtual Agent/chatbot to handle refunds, disputes, and route recommendations conversationally.
- Support for multi-modal journey planning (metro + bus/cab) within the same booking flow.
- Loyalty and season-pass auto-renewal with personalized fare offers.
- Deeper IoT integration with AFC gate hardware for predictive maintenance and crowd-density sensing.

11. APPENDIX

Source Code

Catalog Client Script: Fare Caluculation

```

Catalog Client Scripts
FareCalculation

Script <>

1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5
6   var source = g_form.getValue('starting_from');
7   var destination = g_form.getValue('going_to');
8   var numPassengers = parseInt(g_form.getValue('no_of_passengers'));
9   var groupTicket = g_form.getValue('group_ticket');
10
11   var routeFares = [
12     { source: "7528d891939e4b1013b236befaba1032", destination:
13       "b038d891939e4b1013b236befaba1099", rate: 50 },
14     { source: "7528d891939e4b1013b236befaba1032", destination:
15       "fe38d891939e4b1013b236befaba10a7", rate: 60 },
16     { source: "7528d891939e4b1013b236befaba1032", destination:
17       "b848dc91939e4b1013b236befaba1023", rate: 55 },
18     { source: "7528d891939e4b1013b236befaba1032", destination:
19       "b24810d1939e4b1013b236befaba10a7", rate: 40 }
20   ];
21
22   var perPassengerRate = null;
23   for (var i = 0; i < routeFares.length; i++) {

```

```

Catalog Client Scripts
FareCalculation

Script <>

1 function onChange(control, oldValue, newValue, isLoading) {
19   for (var i = 0; i < routeFares.length; i++) {
20     if (routeFares[i].source == source && routeFares[i].destination ==
21       destination) {
22       perPassengerRate = routeFares[i].rate;
23       break;
24     }
25   }
26
27   if (perPassengerRate == null || isNaN(numPassengers)) {
28     return;
29   }
30
31   var fare = 0;
32
33   if (newValue == '1') {
34     fare = numPassengers * perPassengerRate;
35   } else if (newValue == '2') {
36     fare = 2 * (numPassengers * perPassengerRate);
37   } else {
38     return;
39   }

```

```

Catalog Client Scripts
FareCalculation

Script <>

1 function onChange(control, oldValue, newValue, isLoading) {
30   } else {
31     return;
32   }
33
34   if ((groupTicket == true || groupTicket == 'true') && numPassengers >= 5) {
35     fare = fare - (fare * 0.10);
36   }
37
38   if (newValue == '1') {
39     g_form.setValue('amount_for_single_journey', fare);
40     g_form.clearValue('amount_including_return');
41   } else if (newValue == '2') {
42     g_form.clearValue('amount_for_single_journey');
43     g_form.setValue('amount_including_return', fare);
44   }
45
46   }
47
48   }

```


Catalog Client Script: QR Generation

Script Editor for Catalog Client Scripts: QR Generation

```

1 function onSubmit() {
2   var work = g_form.getValue('what_do_you_want_to_do_today');
3
4   if (work == 2 && g_form.getValue('qr_confirmed') != 'true') {
5     var sysId = g_form.getUniqueValue();
6     var dummyURL = 'https://example.com/ticket?id=' + sysId;
7
8     spModal.open({
9       title: "Scan Your QR Here",
10      widget: "qr_ticket_widget",
11      widgetInput: { ticketUrl: dummyURL }
12    }).then(function() {
13      g_form.setValue('qr_confirmed', 'true');
14      g_form.submit();
15    });
16
17    return false;
18  }
19
20  return true;
21 }

```

Catalog Client Script: Field Validation

Script Editor for Catalog Client Scripts: fieldValidation

Applies on Catalog Tasks ☒

```

1 function onChange(control, oldValue, newValue, isLoading) {
2
3   if (isLoading || newValue == '') {
4     return;
5   }
6
7   if (isNaN(newValue)) {
8     alert('please enter the values in number format only');
9   }
10
11 }
12
13
14
15
16
17

```

Widget: QR Ticket Generation

Widget Editor for QR Ticket Widget

✱ Name: QR Ticket Widget

ID: qr_ticket_widget

Description:

Application: Global

Public: ☐

Roles:

Body HTML template

```

<div>
  <!-- your widget template -->
  <div style="text-align:center; padding:20px;">
    
  </div>
</div>

```

The screenshot shows the 'Widget QR Ticket Widget' editor. The 'Server script' section contains the following code:

```

1 (function() {
2   /* populate the 'data' object */
3   /* e.g., data.table = $sp.getValue('table'); */
4
5 })();

```

The 'Client controller' section contains the following code:

```

1 api.controller = function($scope) {
2   var c = this;
3
4   c.qrUrl = "https://api.qrserver.com/v1/create-qr-code/?size=300x300&data=" +
5     encodeURIComponent(c.data.ticketUrl);
6 };

```

UI Pages: external_qr_api_page

The screenshot shows the 'UI Page external_qr_api_page' editor. The 'Description' field contains 'QR Code'. The 'HTML' section contains the following code:

```

1 <script><![CDATA[
2   (function() {
3     var data = "${sysparm_data}";
4     var decoded = decodeURIComponent(data);
5     var qrApiUrl = "https://api.qrserver.com/v1/create-qr-code/?size=300x300&
6       data=" + encodeURIComponent(decoded);
7     document.getElementById('qr_image').src = qrApiUrl;
8   })();
9
10  function confirmAndSubmit() {
11    if (typeof top.g_form !== 'undefined') {
12      top.g_form.setValue('qr_confirmed', 'true');
13    }
14    if (typeof GlideModal !== 'undefined' && GlideModal.get()) {
15      GlideModal.get().destroy();
16    }
17    if (typeof top.g_form !== 'undefined') {
18      top.g_form.submit();
19    }
20  }

```

GitHub & Project Demo Link

GitHub Repository: <https://github.com/prasannakurri-06/Metro-Ticket-Generating-System.git>